Informe de laboratorio 03

Tema: Programación para el microprocesador INTEL - Instrucciones de transferencia de datos

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza (Teoría grupo B) ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Arq. de computadoras Semestre: IV Código: 1702227

Laboratorio	Tema	Duración
03	Programación para el microprocesador INTEL - Instrucciones de transferencia de datos	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	29/09/2026	15/10/2026

Índice

1. Actividades

1.1. Programa 1: Instrucciones de transferencia de datos

Ahora analice el siguiente código fuente (llamado PRIMER PROGRAMA), ayudándose de los comentarios y del diagrama de flujo que lo acompaña. Intente predecir cada cambio que tendrán los datos hexadecimales en los registros del microprocesador.

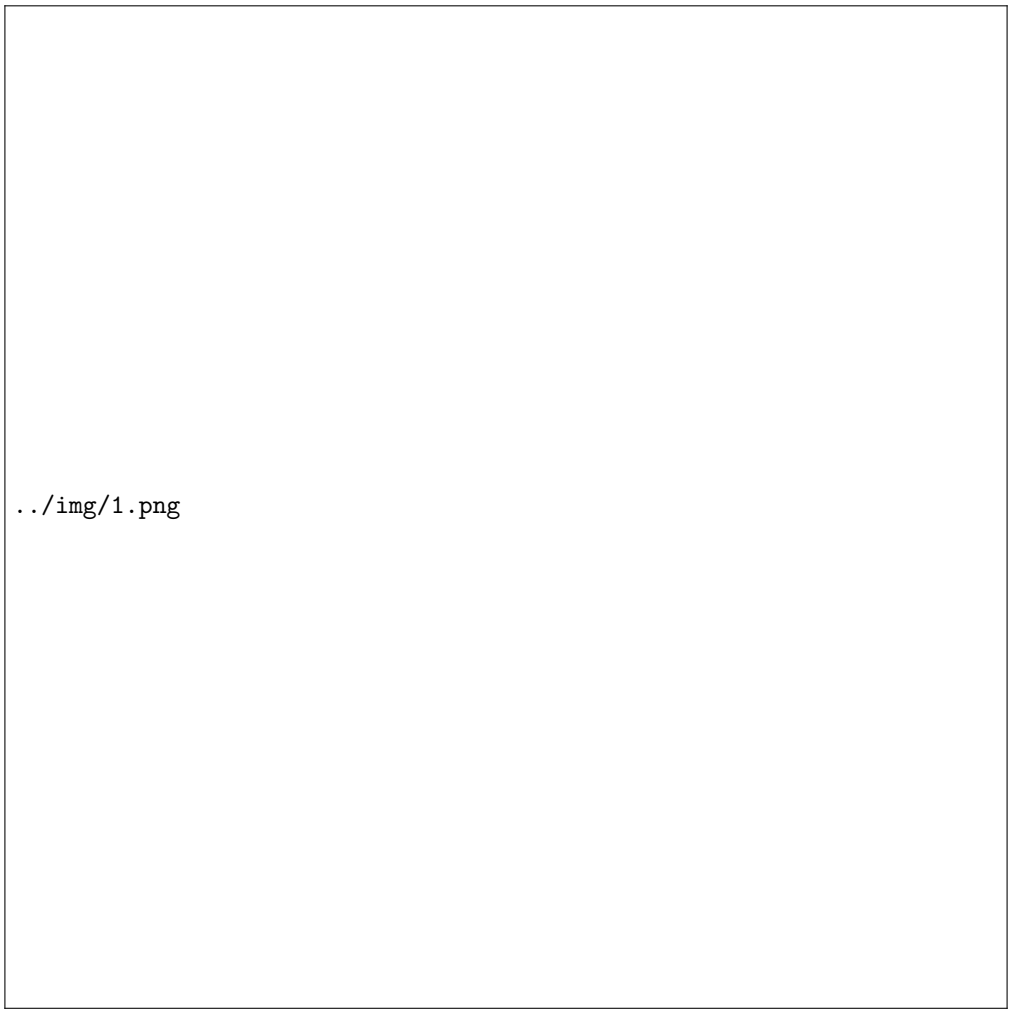


Figura 1: programa1.asm

- Observe que el sistema le asigna al archivo .COM el mismo nombre que el archivo .asm, es decir, programa1.com

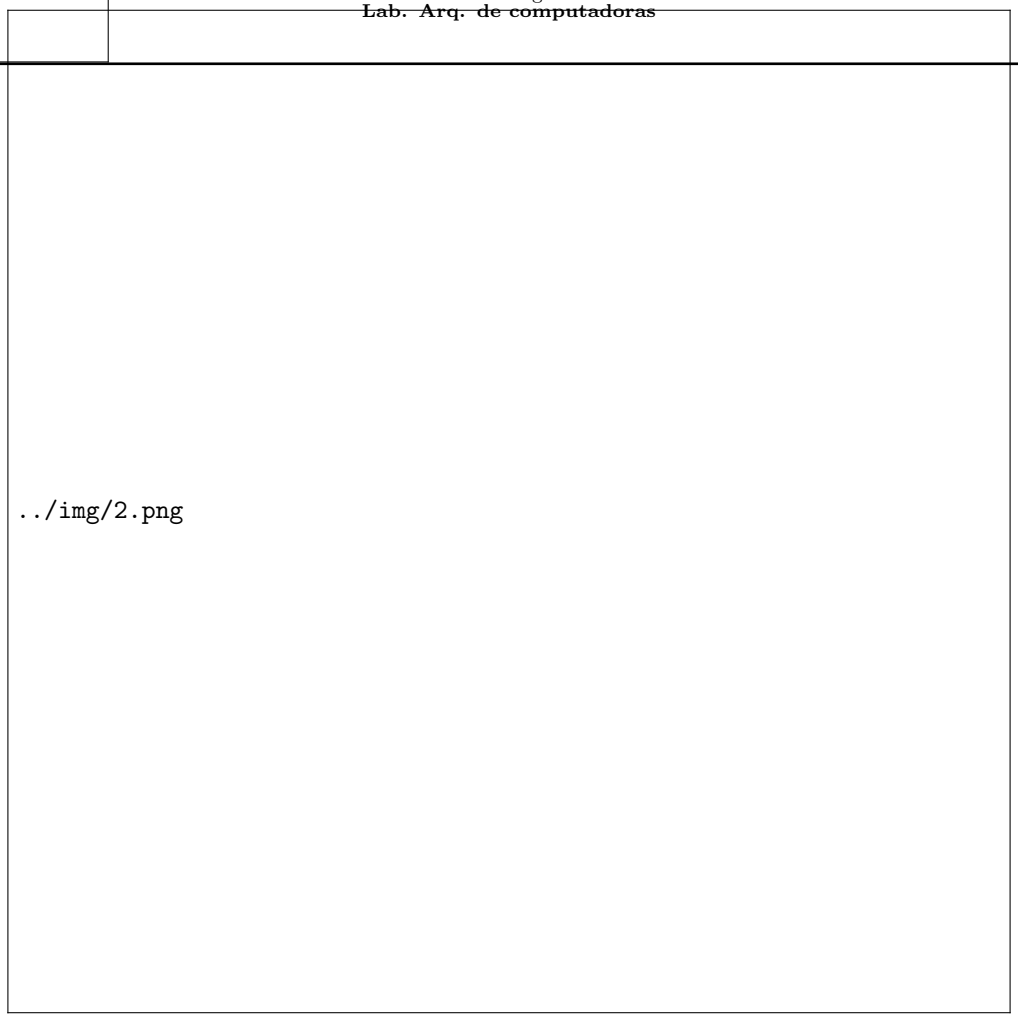


Figura 2: Emulador de programa1.asm

- Copie los datos almacenados en los registros antes de ejecutar el programa:

AX: 0000	BX: 0000	CX: 0000	DX: 0000
CS: 0700	DS: 0700	ES: 0700	SS: 0700
BP: 0000	IP: 0100	SP: FFFE	
DI: 0000	SI: 0000		

Cuadro 1: Datos almacenados antes de ejecutar el programa.

- Ejecute el programa totalmente (RUN) y anote los datos resultantes. Verifique si concuerdan con los datos que usted esperaba.

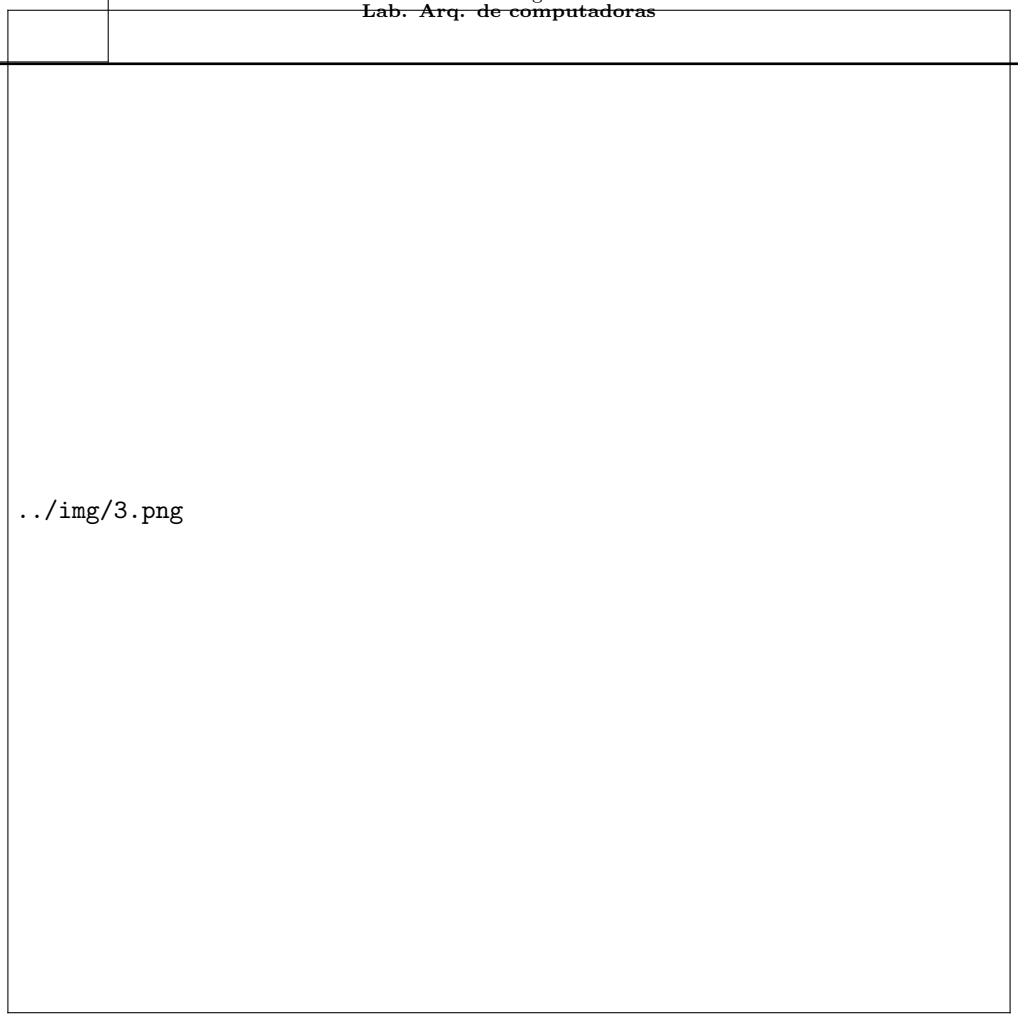


Figura 3: Emulador de programa1.asm

AX: 0064	BX: 0041	CX: ABCD	DX: 198C
CS: F400	DS: 0700	ES: 0700	SS: 0700
BP: 0000	IP: 0154	SP: FFFA	
DI: 0000	SI: 0000		

Cuadro 2: Datos resultantes después de ejecutar el programa.

- Ejecute el emulador con Single step y anote los resultados en la Cuadro.

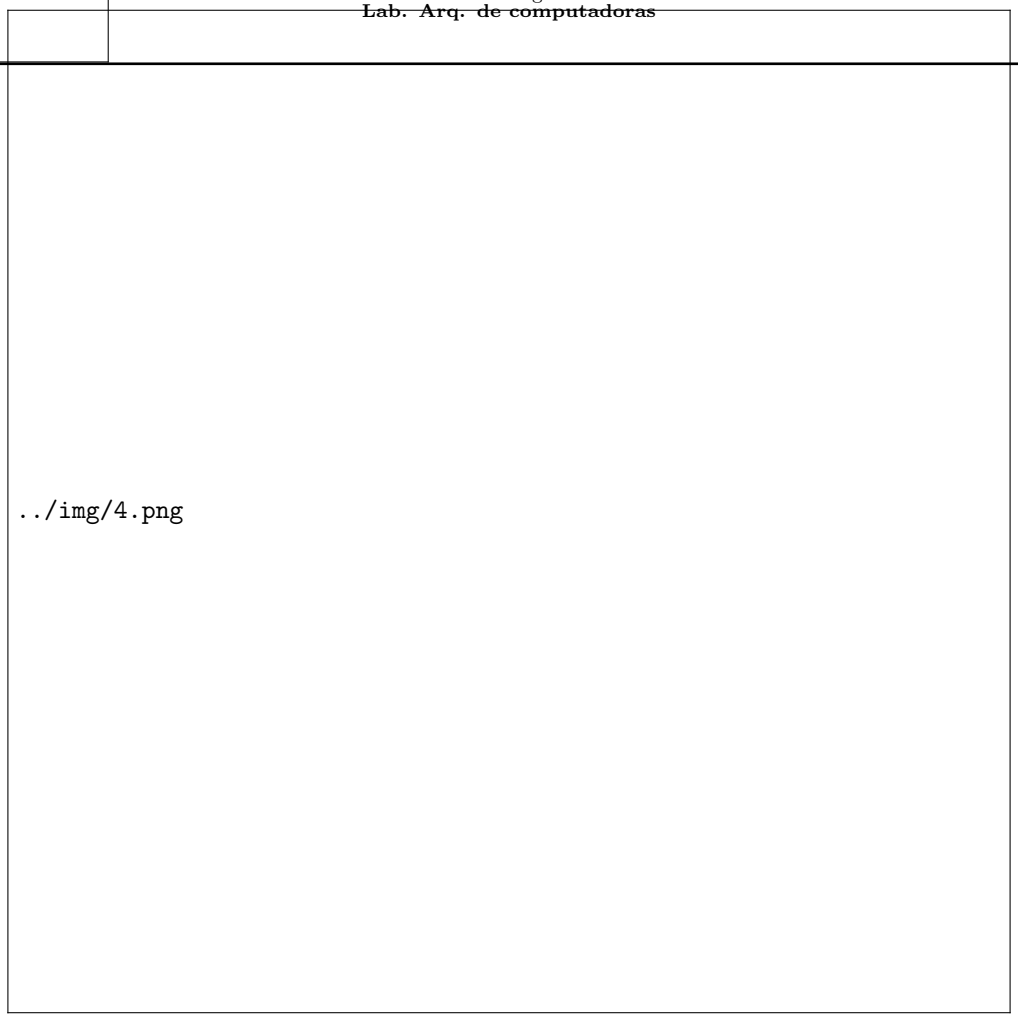


Figura 4: Emulador ejecutado con Single step

Cuadro 3: Programa 1: transferencia de datos

DIRECCIONES		**LENGUAJE DE MÁQUINA**			**LENGUAJE ENSAMBLADOR**
SEGM (CS)	**OFFSET**	**CAMPOS**			**LÍNEA**
0700	0100	B4	7A		mov AH, 7Ah
0700	0102	B3	63		mov BL, 99
0700	0104	B1	CF		mov CL, 11001111b
0700	0106	B8	64	00	mov AX, 100
0700	0109	BB	41	00	mov BX, 78
0700	010C	B9	CD	AB	mov CX, 0ABCDh
0700	010F	BA	8C	19	mov DX, 6540
0700	0112	C3			ret
NOTA: A partir de este punto el programa ejecuta un procedimiento predeterminado llamado Interrupción.					
0700	0000	CD	20		INT 20h
F400	0150	FF	FF		BIOS DI

1.2. Programa 2: Modos de direccionamiento con MOV

¿Cómo el ensamblador se traduce a lenguaje máquina (opcodes, direcciones, tamaño de cada instrucción) y cómo el IP avanza por el programa?

Cuadro 4: Registros antes y después de cada instrucción

Reg.Seg (DS)	**OFFSET (IP)**	**Inicio del programa**	**Antes de 5ª**	**Después de 5ª**	**Después de 6ª**	**Después de 7ª**
0700	**0112h**	90	90	61	61	61
0700	**0113h**	90	90	90	41	41
REGISTRO CX		0011	0011	0011	4161	4161

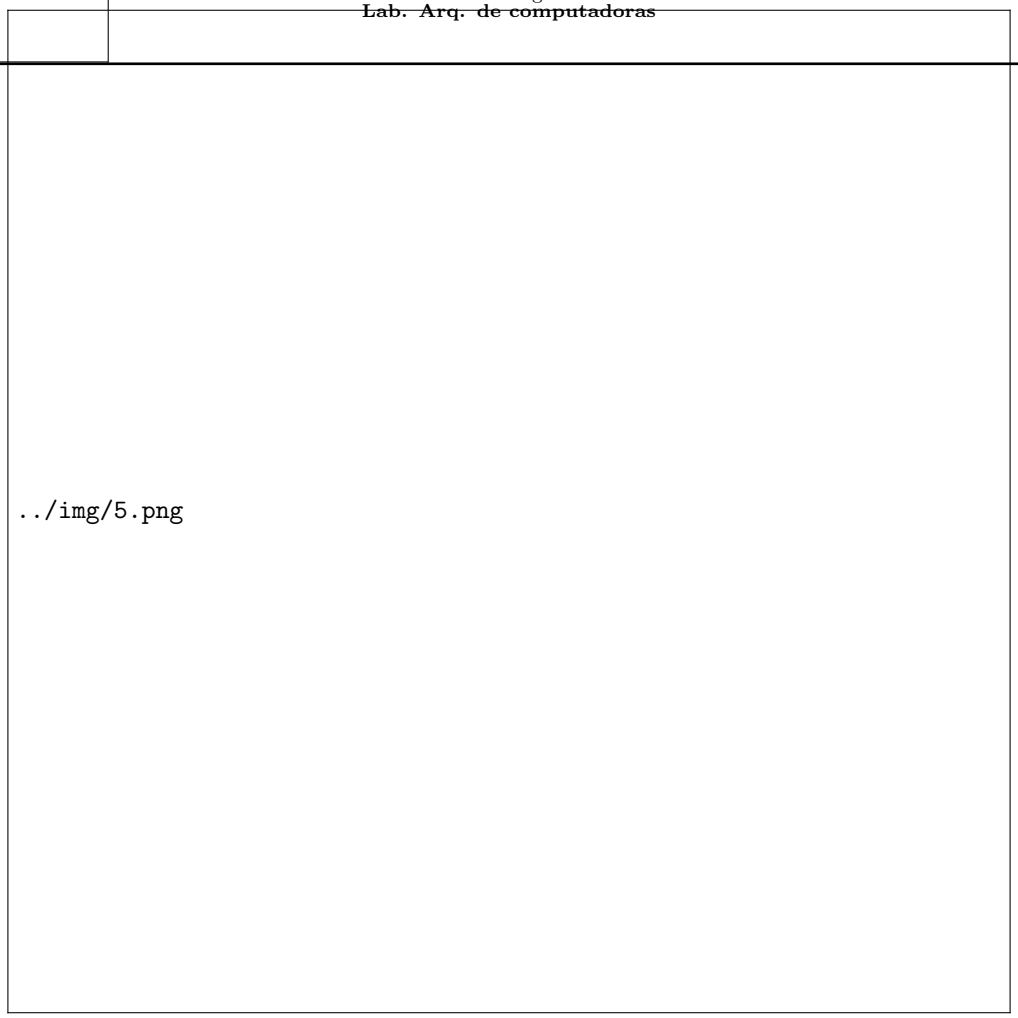


Figura 5: Visualización de cambio de registros y de direcciones de memoria usando Single step

Cuadro 5: Direcciones, lenguaje de máquina y lenguaje ensamblador

DIRECCIONES		LENGUAJE DE MÁQUINA			LENGUAJE ENSAMBLADOR
SEGM (CS)	OFFSET	CAMPOS			LÍNEA
0700	0100	BA	12	01	MOV DX, 0112h
0700	0103	8B	DA		MOV BX, DX
0700	0105	B0	41		MOV AL, 'A'
0700	0107	B4	61		MOV AH, 61h
0700	0109	88	27		MOV [BX], AH
0700	010B	88	47	01	MOV [BX+1], AL
0700	010E	8B	0F		MOV CX, [BX]
0700	0110	C3			RET

1.3. Programa 3: Escritura directa en memoria de video

Escribir directamente en la memoria de video (segmento B800h) para imprimir un carácter con color de fondo y de letra, y entender el byte de atributo.

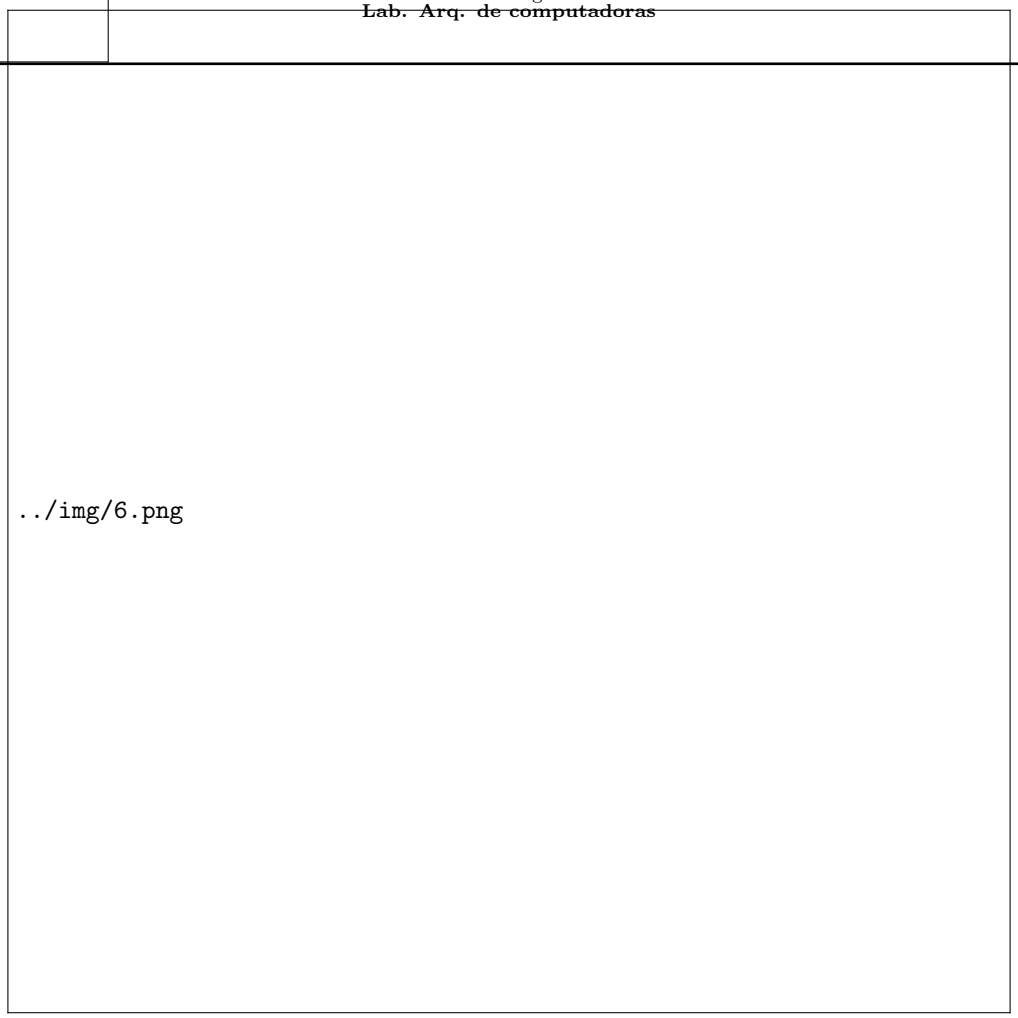


Figura 6: Emulación de programa3.asm

- **Observación:** El programa imprime el caracter 'A' de color blanco y fondo fucsia, ubicado en la 3ra fila y columna 16.

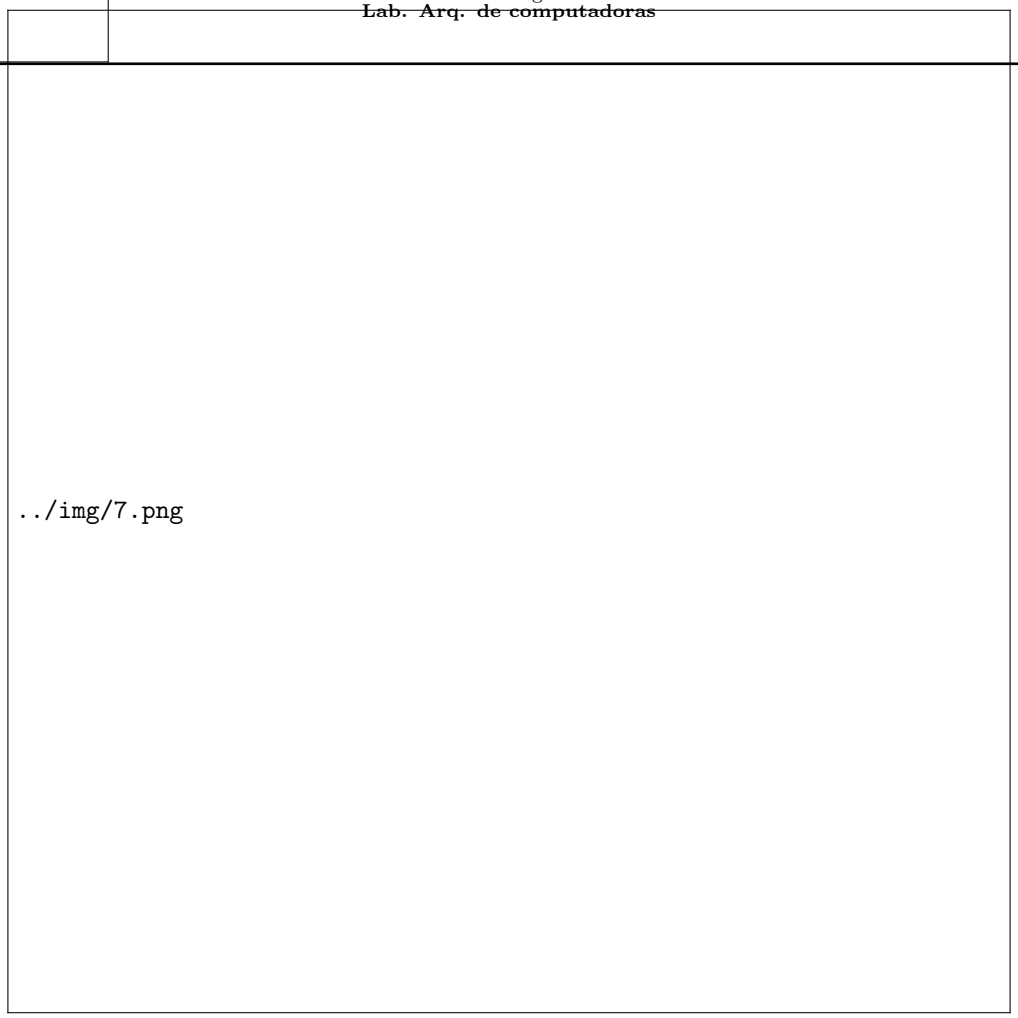


Figura 7: Emulación de programa3(10Eh).asm

- **Observación:** A diferencia del anterior, este cambia de ubicación al caracter, poniéndolo en la 2da fila y columna 56.

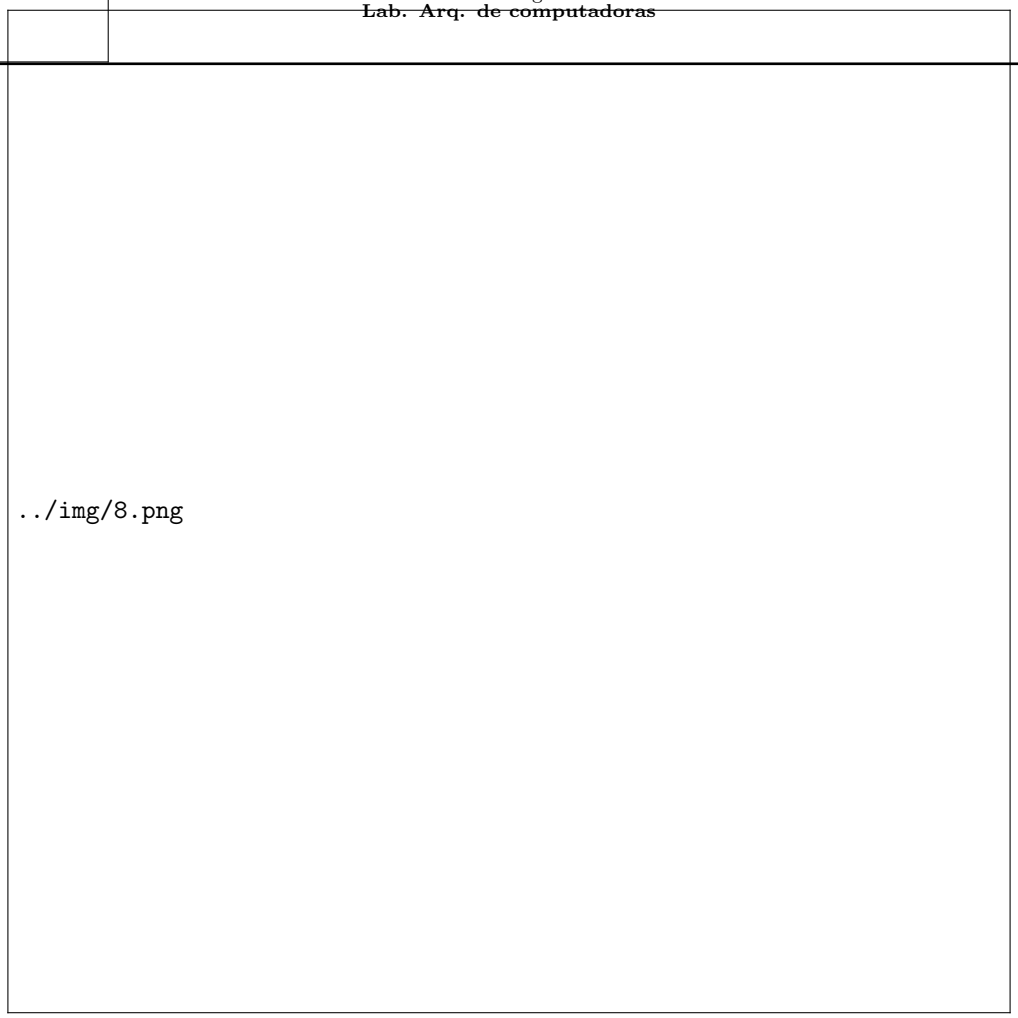


Figura 8: Emulación de programa3(1ch).asm: CH cambiado a 11010000b

- **Observación:** En este caso, el programa cambia solo el color del caracter (negro). Esto lo hace cambiando el valor de CH a 11010000b.

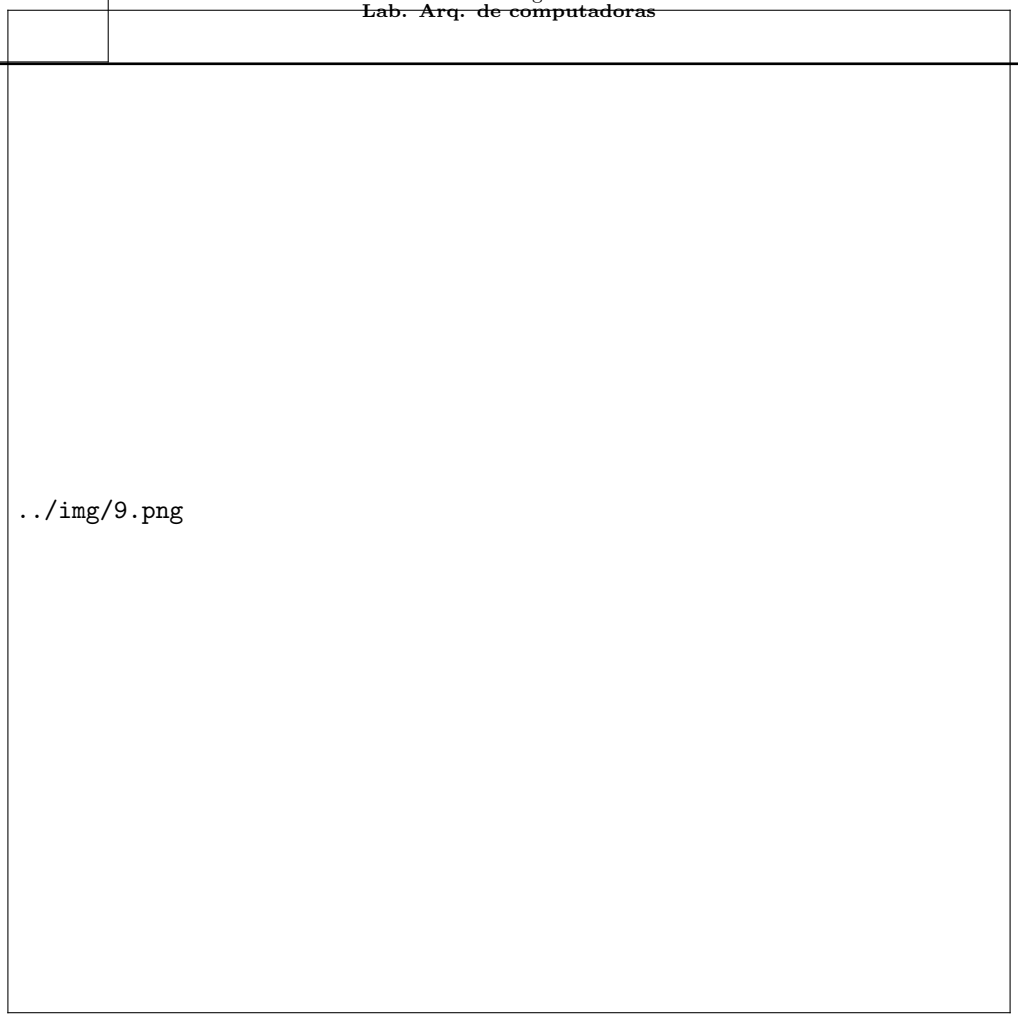


Figura 9: Emulación de programa3(2ch).asm: CH cambiado a 10011110b

- **Observación:** En este caso, el programa cambia el color del fondo del caracter (amarillo) y del caracter mismo(azul claro). Esto lo hace cambiando el valor de CH a 10011110b.

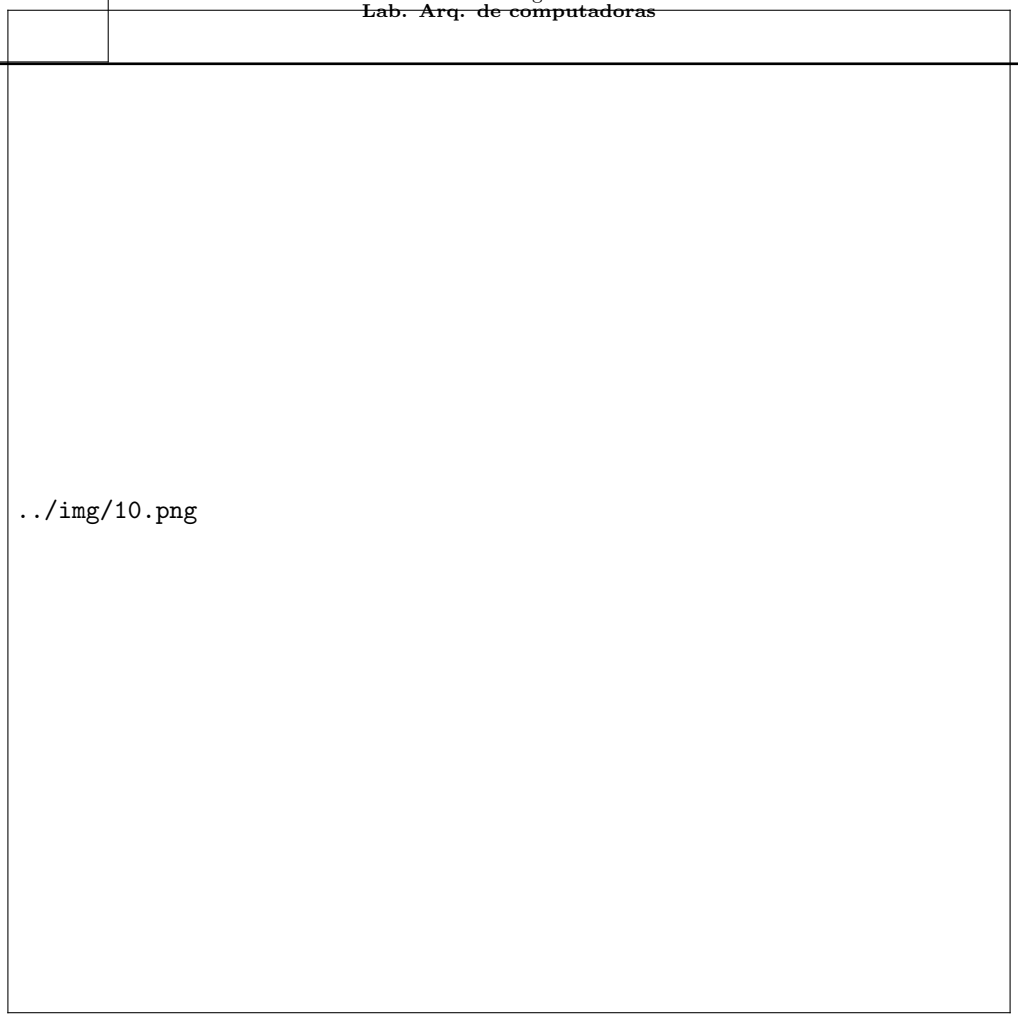


Figura 10: Emulación de programa3(ownCH).asm: CL = 'M', CH = 11111111b, BX = 24Eh

- **Observación:** Para este programa se ha cambiado los 3 valores de CL, CH y BX. Los cambios respectivos son: Ahora el caracter es 'M', CH = 11111111h, es decir, fondo y color del caracter blancos, BX = 24Eh, es decir, el caracter se encuentra en la 4ta fila y en la columna 56.

2. Cuestionario

a) **Primer programa.**

1. **En el encabezado del programa, ¿por qué se incluye la línea `org 100h`?**
   Porque un archivo `.COM` se carga en memoria dejando libres los primeros 256 bytes (100h) del segmento para el PSP (*Program Segment Prefix*). El código comienza entonces en el desplazamiento 0100h. La directiva `ORG` le indica al ensamblador que calcule todas las direcciones a partir de ese punto, y por eso al cargar el programa IP vale 0100.
2. **Al introducir datos en los registros se usan diferentes formatos (bases de numeración), ¿cuántos y cuáles son?**
   Se usaron cuatro formatos: decimal (`99`, `100`, `6540`), hexadecimal (`7Ah`, `0ABCDh`), binario (`11001111b`) y carácter ASCII (`'A'`). En los tres primeros la base se indica con el sufijo (ninguno, `h` o `b`), y el carácter entre comillas se convierte a su código ASCII (41h).

3. **La sintaxis de MOV define tres campos: instrucción, dato1 y dato2. Al moverse los datos, ¿cuál es la fuente y cuál el destino?**
   El formato es `MOV destino, fuente`: *dato1* es el destino y *dato2* es la fuente. Por ejemplo, en `MOV AH, 7Ah` el destino es AH y la fuente es el valor 7Ah. El dato se copia, por lo que la fuente no se modifica.
4. **Al observar la Tabla 1 se nota que a la misma instrucción MOV se le asignan diferentes códigos hexadecimales, ¿a qué se debe esto?**
   El código de operación depende del registro destino y del tamaño del operando. Para cargar un dato inmediato en un registro de 8 bits el código es B0h más el número del registro (B4 para AH, B3 para BL, B1 para CL). Para registros de 16 bits es B8h más el número del registro (B8 para AX, BB para BX, B9 para CX, BA para DX). Es decir, aunque el mnemónico sea el mismo, el procesador necesita un código distinto para cada combinación.
5. **¿Cuál es la función que cumple el registro IP dentro del primer programa?**
   IP (*Instruction Pointer*) contiene el desplazamiento, dentro del segmento de código CS, de la siguiente instrucción por ejecutar. La pareja CS:IP indica al procesador dónde leer. Comienza en 0100h y se incrementa automáticamente según el tamaño de cada instrucción ejecutada, avanzando por el programa hasta llegar a `RET`.
6. **En la columna OFFSET de la Tabla 1 los valores no se incrementan a una razón constante, ¿por qué?**
   Porque las instrucciones no ocupan el mismo número de bytes. Las instrucciones con dato inmediato de 8 bits (por ejemplo `MOV AH, 7Ah`) miden 2 bytes, las de 16 bits (por ejemplo `MOV AX, 100`) miden 3 bytes y `RET` mide 1 byte. Cada offset es el anterior más el tamaño de esa instrucción: 0100, 0102, 0104, 0106, 0109, 010C, 010F y 0112.

b) **Segundo programa.**

1. **Al comparar las dos primeras instrucciones, ¿qué diferencias hay al ejecutarlas, siendo que las dos son MOV?**
   `MOV DX, 0112h` usa direccionamiento inmediato: el dato forma parte de la propia instrucción (3 bytes: BA 12 01) y se carga en DX. `MOV BX, DX` usa direccionamiento por registro: copia el contenido de un registro en otro, no lleva dato inmediato y ocupa solo 2 bytes (8B DA). En ambas solo cambian registros internos y no se accede a la memoria.
2. **En las últimas tres instrucciones se incluyeron corchetes, ¿qué diferencias observó en su ejecución respecto a las primeras dos?**
   Los corchetes indican que el operando es la posición de memoria cuya dirección está en el registro (DS:BX), no el valor del registro. Por eso estas instrucciones sí acceden a la memoria: `MOV [BX], AH` y `MOV [BX+1], AL` escriben en las direcciones 0112h y 0113h, y `MOV CX, [BX]` lee de ellas. El valor de BX no cambia. Además, estas instrucciones llevan un byte extra que codifica el modo de direccionamiento (27, 47 01 y 0F), y `[BX+1]` agrega un desplazamiento, por lo que mide 3 bytes.
3. **¿Por qué al ejecutar la séptima instrucción los datos se almacenan en CX en ese orden específico?**
   Porque el 8086 almacena los datos de 16 bits en formato *little endian*: el byte menos significativo va en la dirección menor y el más significativo en la siguiente. Como en 0112h está 61h y en 0113h está 41h, al leer con `MOV CX, [BX]` el primer byte (61h) pasa a CL y el segundo (41h) a CH, quedando CX = 4161h.

c) **Tercer programa.**

1. **Al ejecutar el programa, a medida que hacía las modificaciones, ¿qué efecto concreto logra el cambio en el registro BX?**
   BX contiene el desplazamiento dentro de la memoria de video (segmento B800h) donde se escribe el carácter y su atributo, así que determina la **posición** en la pantalla. Cada fila de 80 columnas ocupa 160 bytes y cada celda 2 bytes. Con 15Eh la letra aparece en la fila 3, columna 16, y con 10Eh en la fila 2, columna 56.

2. **Al modificar CH, ¿qué bits específicos se modificaron para lograr cambios en el fondo y color de los caracteres?**
   CH es el byte de atributo. Los cuatro bits bajos (bits 0 a 3) definen el color de la letra y los cuatro bits altos (bits 4 a 7) definen el color de fondo. Por ejemplo, al pasar de `11011111b` a `11010000b` solo cambiaron los bits bajos y la letra pasó de blanco a negro, manteniendo el fondo.

3. Investigación complementaria

1. **¿Qué son los modos de direccionamiento?**
   Son las distintas formas en que una instrucción indica dónde se encuentra el dato (operando) con el que va a trabajar o dónde se almacenará el resultado. El operando puede estar incluido en la propia instrucción, en un registro o en una posición de memoria, y cada modo define cómo calcular esa ubicación. En el caso de la memoria, el procesador calcula una *dirección efectiva* (un desplazamiento) que combina con el registro de segmento correspondiente para obtener la dirección física.

2. **¿Cuántos y cuáles modos de direccionamiento tiene el microprocesador 8086?**
   Para el acceso a datos, la bibliografía suele describir siete modos (algunos autores agrupan o cuentan distinto, por lo que la cifra puede variar):

   - **Por registro:** el operando está en un registro, por ejemplo `MOV BX, DX`.
   - **Inmediato:** el dato forma parte de la instrucción, por ejemplo `MOV AH, 7Ah`.
   - **Directo:** la instrucción contiene la dirección (desplazamiento) del dato en memoria, por ejemplo `MOV AX, [0112h]`.
   - **Indirecto por registro:** la dirección está en BX, SI, DI o BP, por ejemplo `MOV [BX], AH`.
   - **Relativo a base o a índice:** dirección = registro (BX, BP, SI o DI) más un desplazamiento, por ejemplo `MOV [BX+1], AL`.
   - **Indexado con base:** dirección = registro base (BX o BP) más registro índice (SI o DI), por ejemplo `MOV AX, [BX+SI]`.
   - **Indexado con base y desplazamiento:** suma de base, índice y un desplazamiento, por ejemplo `MOV AX, [BX+SI+4]`.

   Además existen modos propios de otras instrucciones: direccionamiento relativo para saltos y llamadas (`JMP`, `CALL`), direccionamiento de puertos de E/S (`IN`, `OUT`) y el de cadenas (`MOVS`, `LODS`), que usa DS:SI y ES:DI de forma implícita.

3. **¿Qué modos de direccionamiento posee la instrucción MOV?**
   `MOV` admite todos los modos de direccionamiento de datos descritos arriba: por registro, inmediato, directo, indirecto por registro, relativo a base o índice, y los indexados con base (con o sin desplazamiento). En las prácticas se utilizaron el modo inmediato (`MOV DX, 0112h`), por registro (`MOV BX, DX`), indirecto por registro (`MOV [BX], AH`) y relativo a base (`MOV [BX+1], AL`). Tiene restricciones: no permite mover directamente de memoria a memoria, no admite cargar un dato inmediato en un registro de segmento (por eso se usa `MOV AX, 0B800h` y luego `MOV DS, AX`), no permite mover un registro de segmento a otro ni escribir en CS.

4. **¿Qué diferencias hay entre las instrucciones XCHG y MOV?**

   - **Operación:** `MOV` copia el contenido de la fuente en el destino y la fuente no cambia. `XCHG` intercambia los contenidos de ambos operandos, por lo que los dos se modifican.
   - **Operandos permitidos:** `XCHG` no admite datos inmediatos ni registros de segmento, y sus dos operandos deben ser del mismo tamaño. Uno de ellos debe ser un registro (no puede haber memoria con memoria). `MOV` sí admite datos inmediatos y registros de segmento (con las restricciones indicadas).

- **Equivalencia:** para hacer con `MOV` lo que hace un solo `XCHG` se necesitan tres instrucciones y un registro auxiliar (`MOV AUX, A`; `MOV A, B`; `MOV B, AUX`).
- **Banderas:** ninguna de las dos instrucciones modifica el registro de banderas.

4. Conclusiones

- El emulador EMU8086 permitió relacionar el código fuente en ensamblador con su representación en lenguaje máquina y observar cómo cambian los registros del 8086 después de cada instrucción, tanto con la ejecución paso a paso como con la ejecución completa.
- En el primer programa se comprobó que cada instrucción `MOV` se codifica con un código de operación distinto según el registro y su tamaño, y que IP avanza según los bytes que ocupa cada instrucción, lo que explica que los desplazamientos no crezcan a un ritmo constante.
- En el segundo programa se verificó la diferencia entre mover datos entre registros y acceder a la memoria mediante direccionamiento indirecto, y se comprobó el almacenamiento en formato *little endian*: el byte menos significativo se guarda en la dirección menor, lo que explica el valor 4161h en CX.
- En el tercer programa se observó que escribir en la memoria de video (segmento B800h) permite mostrar un carácter en pantalla. El registro BX determina su posición y el byte de atributo en CH define el color de fondo (bits altos) y el color de la letra (bits bajos).
- Se reconocieron los modos de direccionamiento básicos del 8086 (inmediato, por registro, indirecto y relativo a base) y la forma en que se calcula la dirección física a partir del segmento y el desplazamiento.
- Se aprendió el flujo de trabajo completo con el emulador: escribir el código fuente, compilarlo para generar el archivo `.COM`, cargarlo en el emulador y analizar su ejecución.

A. Anexos

A.1. Tabla de colores (4 bits)

Bin	Hex	Color	Bin	Hex	Color
0000	0	Negro	1000	8	Gris oscuro
0001	1	Azul	1001	9	Azul claro
0010	2	Verde	1010	A	Verde claro
0011	3	Cian	1011	B	Cian claro
0100	4	Rojo	1100	C	Rojo claro
0101	5	Magenta	1101	D	Magenta claro
0110	6	Marrón	1110	E	Amarillo
0111	7	Gris claro	1111	F	Blanco